AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_C_ingenjörsjobb.pdf Version : V1.0	

LOW LEVEL C-PROGRAMMING AND MICROCOMPUTER ARCHITECTURE – GRAFISK VISUALISERING FÖR ACCELEROMETER

Darwin Rengifo

2018-07-20

Sammanfattning: Detta dokument beskriver utvecklingen av ett system som logga kretskortets accelerometer. Denna loggning ska visas på ett grafiskt sätt på en VGA-skärm. I rapporten behandlas krav, förstudie, testprotokoll, konstruktion, verifiering, validering, analys och optimering.

Innehåll

LOW LEVEL C-PROGRAMMING AND MICROCOMPUTER ARCHITECTURE – GRAFISK VISUALISERING FÖR ACCELEROMETER 1

1. INLEDNING 4
2. KRAVSPECIFIKATION 4
3. BESKRIVNING AV ÄLDRE PROJEKT 9
4. BESKRIVNING AV KORTET DE10-LITE 10
5. BESKRIVNING AV ACCELEROMETERN 11
6. PROJEKTPLAN 11
7. TESTPROTOKOLL 12
8. KONSTRUKTIONSBESKRIVNING 13

8.1. Mjukvara 13
8.1.1. Systemarkitektur 13
8.1.2. Delsystem 15
8.1.3. Drivrutiner 16
8.1.3.1. VGA färger 16
8.1.3.2. VGA-drivrutiner 16
8.1.3.3. Timmer-drivrutiner 18
8.1.3.4. SPI-drivrutiner 18
8.2. Hårdvara 19
8.2.1. Systemarkitektur 19
8.2.2. Minneskarta 19
8.3. Delsystem 20
8.3.1. PLL 20
8.3.2. On-Chip RAM : onchip_ram 20
8.3.3. Timer IP : TIMER_HW_IP_0 20
8.3.4. System ID : sysid_qsys_0 20
8.3.5. Accelerometer SPI : accelerometer_spi_0 20
8.3.6. JTAG UART : jtag_uart 20
8.3.7. SDRAM Controller : sdram_controller 20
8.3.8. LED_PIO : pio_lkeds_out 20
8.3.9. BUTTON_PIO : pio_buttons_in 20
8.3.10. VGA IP : DE10_Lite_VGA_IP_0 20

8.4. Accelerometer (G-sensor) 21
9. VERIFIERING AV TESTPROTOKOLL 21
10. VALIDERING AV TESTPROTOKOLL 22
11. ANALYS FÖRE OPTIMERING 24
12. OPTIMERING 25
13. ANALYS EFTER OPTIMERING 26
14. SAMMANFATTNING 27
15. REFERENSER 27

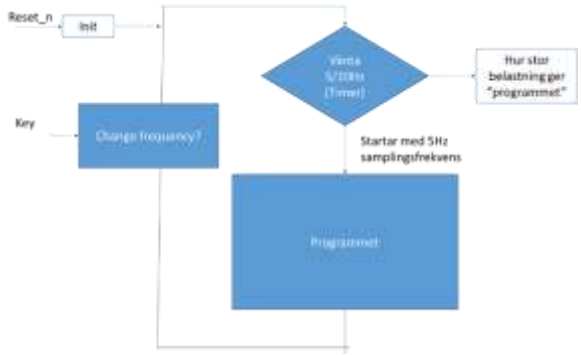
1. INLEDNING

Företaget TEIS har fått ett uppdrag att konstruera ett system som logga kretskortets accelerometer. Loggningen ska visas på ett enkelt ”informativt” sätt på en VGA-skärm. Kunden har även beställt drivrutiner och en SW- manual och den ska beskriva VGA och accelerometerns mjukvarudrivrutiner.

2. KRAVSPECIFIKATION

Tabell 1. Kravspecifikation

Krav	**Beskrivning**	**Utfört Ja/nej**
	Förstudie	
Krav_1	Beskriv kort och ge förbättringskommentarer på ett äldre projekt ur low level C-synvinkel, se ”archive” i kursen Konstruktionsmetodik och teknisk dokumentation. (Eget kapitel)	Ja
Krav_2a	Beskriv kortet som ska användas till projektet. (Eget kapitel)	Ja
Krav_2b	Beskriv mikrodatorarkitekturen, systemarkitekturen, alla I/O komponenterna, minneskartan m.m. (Eget kapitel)	Ja
Krav_2c	Beskriv hårdvaran med alla I/O (VGAn, accelerometern) för en programmerare. (Eget kapitel).	Ja
	Företagets regelkrav	
Krav_3	TEIS måste regler och riktlinjer för C ska följas, se Regler och Riktlinjer för TEIS.	Ja
	Konstruktions krav	

Krav_4a	Arkitektur krav: Programmet ska ”exekveras” med 5 eller 10Hz. Denna mekanism styrs med timern för att få exaktare samplingstid (max +/- 5% fel). Det ska vara möjligt att räkna ut hur lång tid programmet väntar på nästa sampel (ska användas i analysen). Denna del ska vara tydligt beskriven i den tekniska rapporten.  **Figur 2: Programmets arkitektur** SW-systemet kan sägas befinna sig i två tillstånd: • ”Ideal” (väntar), när den väntar på att starta nästa sampel. • ”Running” (exekverar ”programmet”); läser av accelerometern, ritar ut på skärmen osv. Detta är ett enkelt operativsystem.	Ja
	Programmet kan vara uppbyggt på det sättet konstruktören anser vara lämpligast.	Ja
Krav_4b	VGA-skärmen ska först presentera konstruktörens namn och datum. (init)	Ja
Krav_4c	VGA-skärmen ska kunna visa värden med samplingsfrekvenser 5 eller 10Hz i realtid. Värdena ska inte sparas.	Ja
Krav_4d	Visning av accelerometerns värde kan vara de sista bitarna i värdet som erhålls från accelerometern. Ingen noggrannhet krävs. Krav är att alla tre axlarna ska visas, antingen som en graf, staplar eller siffror, det är valfritt.	Ja
Frivillig	Frivillig: Konstruktören kan använda lysdioderna, knapparna och 7 seg displayerna valfritt, t.ex. för att visa processorbelastning.	Ja
Krav_4e	Två konstruktioner ska utföras: 1) inte optimerad och därefter analyseras.	Ja

	2) Därefter ska en optimering göras och en ny analys av systemet utföras.	
Krav_4f	Följande VGA drivrutiner är beställda: En kort SW-manual för VGA-drivrutinerna (exempel se Sierra SW-manual). Koden ska vara robust skriven (följ TEIS regler och riktlinjer). Kunden har också begärt att alla typerna i anropen ska vara exakt definierade, som t.ex. ”alt_u16” (inga ”int”). Funktion**:** print_pix(unsigned int x,unsigned int y,unsigned int rgb); Funktionsbeskrivning: Skriver en pixel med färgen rgb på koordinaten (x, y). Funktion: print_hline(unsigned int x_start, unsigned int y_start, unsigned int len, unsigned int RGB); Funktionsbeskrivning: Skriver en horisontell linje med färgen rgb och med längden len vilken startar på koordinaten (x_start, y_start). Funktion**:** print_vline(unsigned int x_start, unsigned int y_start, unsigned int len, unsigned int RGB); Funktionsbeskrivning: Skriver en vertikal linje med färgen rgb och med längden len vilken startar på koordinaten (x_start, y_start). Funktion: print_char(unsigned int x,unsigned int y,unsigned int rgb,unsigned int BG_RGB,char Character); Funktionsbeskrivning: Skriver tecknet ”Character” med färgen rgb och med bakgrundsfärgen ”BG_RGB” på koordinaten (x, y). Implementera minst de tecken som	Ja

	behövs för att skriva den texten som ska visas på skärmen. Funktion: read_pixel_ram_int(unsigned int x_start, unsigned int y_start); return: unsigned pixel_data (obs! enbart de sista tre bitarna är intressanta). Funktionsbeskrivning: Läser pixel_data från pixel RAM (3 bitar, RGB) från adress x och y (beräknat från x och y). Funktion: print_circle(unsigned int radie, unsigned int x_centrum, unsigned int y_centrum, unsigned int rgb); Funktionsbeskrivning: Skriver en cirkel med radien "radie" och färgen" rgb" på mittkoordinaten (x, y). Cirkeln ska fyllas med samma färg. Funktion: clear_screen(int rgb); Funktionsbeskrivning: Denna funktion rensar skärmen genom att RGB-värdet skrivs till alla pixlar på skärmen. Frivillig: Funktion: print_str(alt_u16 x_start, alt_u16 y_start,alt_u8 rgb,char *str) Funktionsbeskrivning: Print_str skriver ut en sträng på skärmen bokstav för bokstav med hjälp av print_char.	
Analyskrav		
Krav_5	Analysen utförs i två steg. I det första steget analyseras koden före optimering och det andra steget efter optimering. Det som ska analyseras är minst: • Kodstorlek • Min- och maxtiden för följander: o clear_screen o print_circle o print_pix • min/max och % för hur lång tid programmet är i Ideal (väntar på nästa sampel) för den inte optimerade konstruktionen, den optimerade och de två olika frekvenserna.	Ja
Optimeringskrav		
Krav_6	Optimera på alla sätt som TEIS träning har tränat dig på och andra sätt som du finner ut. Skriv koden smart, styr verktygen, styr koden till minnen nära CPUn o.s.v.	Ja

Rapportkrav (konstruktionsrapport)		
Krav_7	Utgå gärna från Rapportmall_TEIS, som ligger på Itslearning i kursen Konstruktionsmetodik och teknisk dokumentation. Kunden kräver en standardiserad teknisk rapport med filnamn ”fornamn_efternamn_C_ingenjorsjobb”. Följande ska finnas med i slutrapporten: • Titel och en kort sammanfattning • Kravspecifikation, eget kapitel (krav) • Beskrivning av ett äldre projekt med några kommentarer om förbättringar för att förstå konstruktionen, eget kapitel (krav) • Beskrivning av kortet eget kapitel (krav). • Beskrivning av accelerometern, eget kapitel (krav). • Projektplan, aktiviteter, budgeterad och verklig kostnad (500 SEK/tim), eget kapitel (krav) • Testprotokoll för validering, grundat på kravspecifikationen. Protokollet behöver inte vara komplett, det räcker att visa hur det ska utföras och det behöver bara visas på systemnivå (inte på subnivåer), eget kapitel. (krav) • Konstruktionsbeskrivning, eget kapitel o Se även Konstruktions krav o Software ❖ System arkitektur (”top-down”) ❖ Delsystem ❖ Drivrutiner (alla) o Hardware ❖ System arkitektur (”top-down”) ❖ Delsystem ❖ Sensorer • Verifiering, Utgår i detta projekt • Validering, eget kapitel o Visa med resultat från testprotokollet o (validering) o Kombineras med en kort film som beskriver för en utomstående vad som genomförts (visa gärna en systemarkitektur) och demonstrera att det fungerar (en länk till YouTube), som en kort presentation av hela jobbet.	Ja

	• Analys före optimering, eget kapitel o Se analys krav • Optimering, eget kapitel o Dokumentera alla settings o Länk direktiv • Analys efter optimering, eget kapitel o Se analys krav o Beskriv vad som är ändrat • Sammanfattning, kända buggar och framtida förbättringar • Referenser • Eventuellt bilagor	
Leveranskrav		
Krav_8	Leveransen ska innehålla tre foldrar: • *Konstruktionsbeskrivning* o Konstruktionsrapport Filnamn: fornamn_efternamn_C_ingenjorsjobb (pdf, word) • *Konstruktionsdokument* o software mappen med applikation och BSP o Alla IP-komponenter (IP mapp) o SOF- och SOPCINFO-filen • Diverse	Ja
Krav_9	Leveransen ska ske till plattformen ltslearning. Namnet på filen ska vara ”fornamn_efternamn_C_ingenjorsjobb_x.zip”. Där x motsvarar version. Sista leveransdag se kurs schema (för VG).	Ja

3. BESKRIVNING AV ÄLDRE PROJEKT

”TEIS - Acceleration, ljus samt temperatur” av Martin Rydberg

Projektet förklarar tillvägagångssättet för att logga mätvärden från en accelerometer, ljussensor samt en termometer på ett BeMicro Max 10 kortet.
För att mätdata från sensorerna på BeMicro kortet skall kunna presenteras på VGA skärmen behövs ett VGA-interfacekort som ansluts till PMOD kontakt A/B.

Förslag på förbättringar

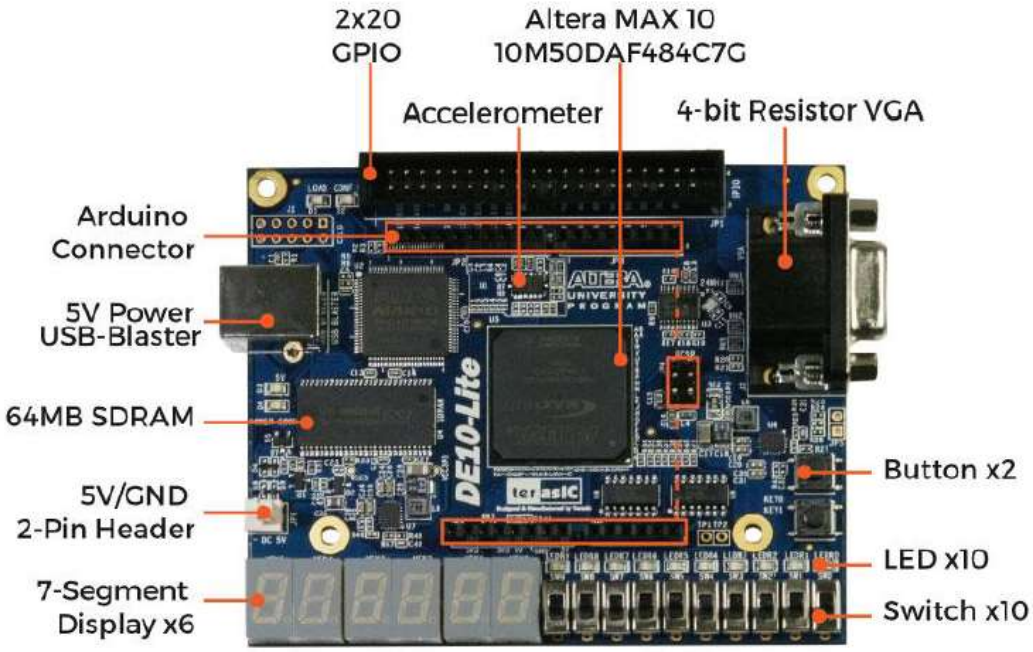
- ❖ Förbättring av visualisering av mätvärden
  För att få en bättre visualisering av arbetet kan det visas både accelerometer, ljussensor samt en termometer på separat sätt i varje segment av skärmen även programmet kan skriver ut på skärmen de absoluta mätvärdena.

4. BESKRIVNING AV KORTET DE10-LITE

Denna kort har många funktioner som gör det möjligt för användare att implementera ett brett utbud av konstruerade kretsar, från enkla kretsar till olika multimedia projekt. Den följande hårdvaran finns på tavlan:

Tabell 2. Minnesmängd [1]

DE10-Lite	Beskrivning
FPGA Device	• MAX 10 10M50DAF484C7G Device • Integrated dual ADCs, each ADC supports 1 dedicated analog input and 8 dual function pins • 50K programmable logic elements • 1,638 Kbit M9K Memory • 5,888 Kbits user flash memory • 144 18 × 18 Multiplier • 4 PLLs
Programming and Configuration	On-Board USB Blaster (Normal type B USB connector)
Memory Device	64MB SDRAM, x16 bits data bus
Sensor	3-axis Accelerometer
Expansion Connectors	• One 2x20 GPIO Connector(voltage levels: 3.3V) • Arduino Uno R3 Connector, including six ADC channels
Display	4-bit Resistor VGA
Switches	10 Slide Switches
Buttons	2 Push Buttons
LEDs	10 LEDs
7-Segment Display	Six 7-Segments Display
Power	5V DC input



Figur 1. Placering av kontakter och nyckelkomponenter av kortet DE10-Lite [1]

5. BESKRIVNING AV ACCELEROMETERN

En accelerometer är en enhet som mäter acceleration i en riktning, vilket är hastighetsförändringen av objektet i G-krafter (g). En G-kraft är ekvivalent med 9.8 m/s2, men detta varierar något beroende på geografisk plats.

Accelerometrar är användbara för att varnar vibrationer i system eller för vägledningstillämpningar.

Accelerometrar är elektromekaniska anordningar som upptäcker acceleration krafter, statiska eller dynamiska. Statiska krafter innefattar gravitation, medan dynamiska krafter kan innefatta vibrationer och rörelse.

Accelerometrar kan mäta accelerationen i en, två eller tre axlar. De av tre axlar är vanligare.

6. PROJEKTPLAN

Aktivitets- och tidplan för ingenjörsjobbet kan ses i Tabell 3. Totalt uppskattas ingenjörsjobbet ta 80 timmar/500kr(timme), vilket ger en lönekostnad på 40 000 SEK.

Tabell 3. Aktivitets- och tidsplan för ingenjörsjobbet

Aktivitet	Veckor				Timmar(h)	Kostnad(kr)
	28	29	30	31		
Kravspec./förstudie	13				13	6 500
Testprotokoll/ Drivrutiner	4	4	1		9	4 500

Arkitekturbeskrivning	5	3			8	4 000
Programmering av visuellt gränssnitt		6	12	10	28	14 000
Verifiering			2	2	4	2 000
Validering				2	2	1 000
Buggrättning			6	3	9	4 500
Slutrapport			8	12	20	10 000
Summa tid	22	13	29	29	80	40 000

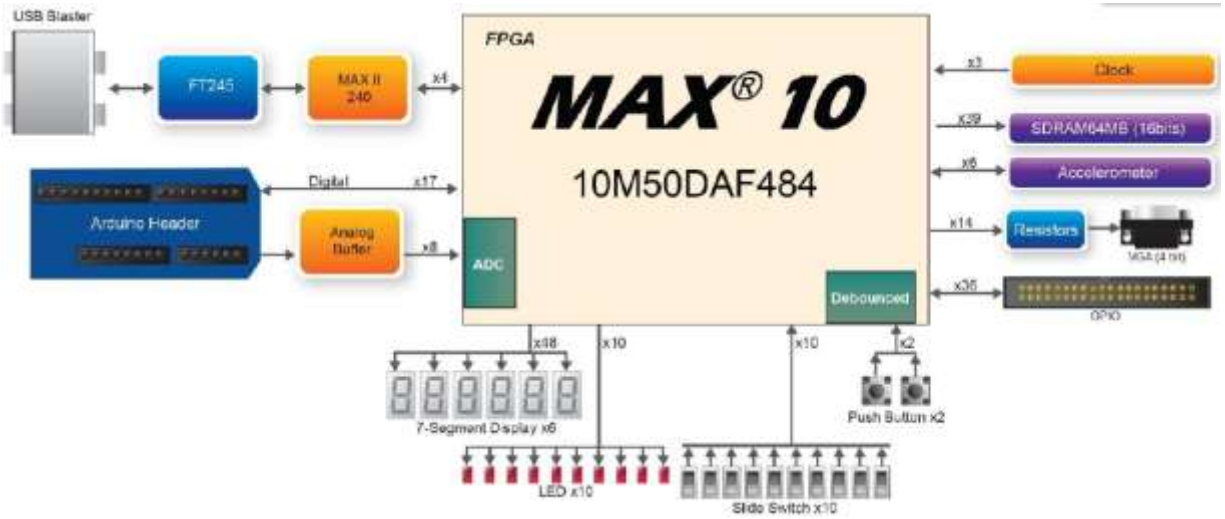
7. TESTPROTOKOLL

Tabell 4. Testprotokoll

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset av systemet. Hela skärmen skall rensas vid uppstart av programmet.	Funktionen clearScreen(0) används för att skriva färgvärdet svart (0) till hela grafikminnet.	
2	Presentation av projektet visas efter att skärmen rensats vid uppstart.	Introduktionsskärm med en kort presentation visas efter uppstart.	
3	Knapp 0 skall skifta mellan ”presentations”-läge och ”loggnings”-läge.	Skärmen visar presentations”-läge eller ”loggnings”-läge.	
4	Tryck på knapp 1 en gång för att öka samplingsfrekvens 5 Hz.	Debugutskrift visar klockcykler som ger ca 5 Hz frekvens avläsning av accelerometer.	
5	Tryck på knapp 1 en gång för att öka samplingsfrekvens 10 Hz.	Debugutskrift visar klockcykler som ger ca 10 Hz frekvens avläsning av accelerometer	
6	Tryck på knapp 1 en gång för att minska samplingsfrekvens 5 Hz.	Debugutskrift visar klockcykler som ger ca 5 Hz frekvens avläsning av accelerometer.	

8. KONSTRUKTIONSBESKRIVNING

Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggd kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet, VGA komponent, PIO enheter för tryckknappar och lysdioder, SPI komponent för avläsning av accelerometer.

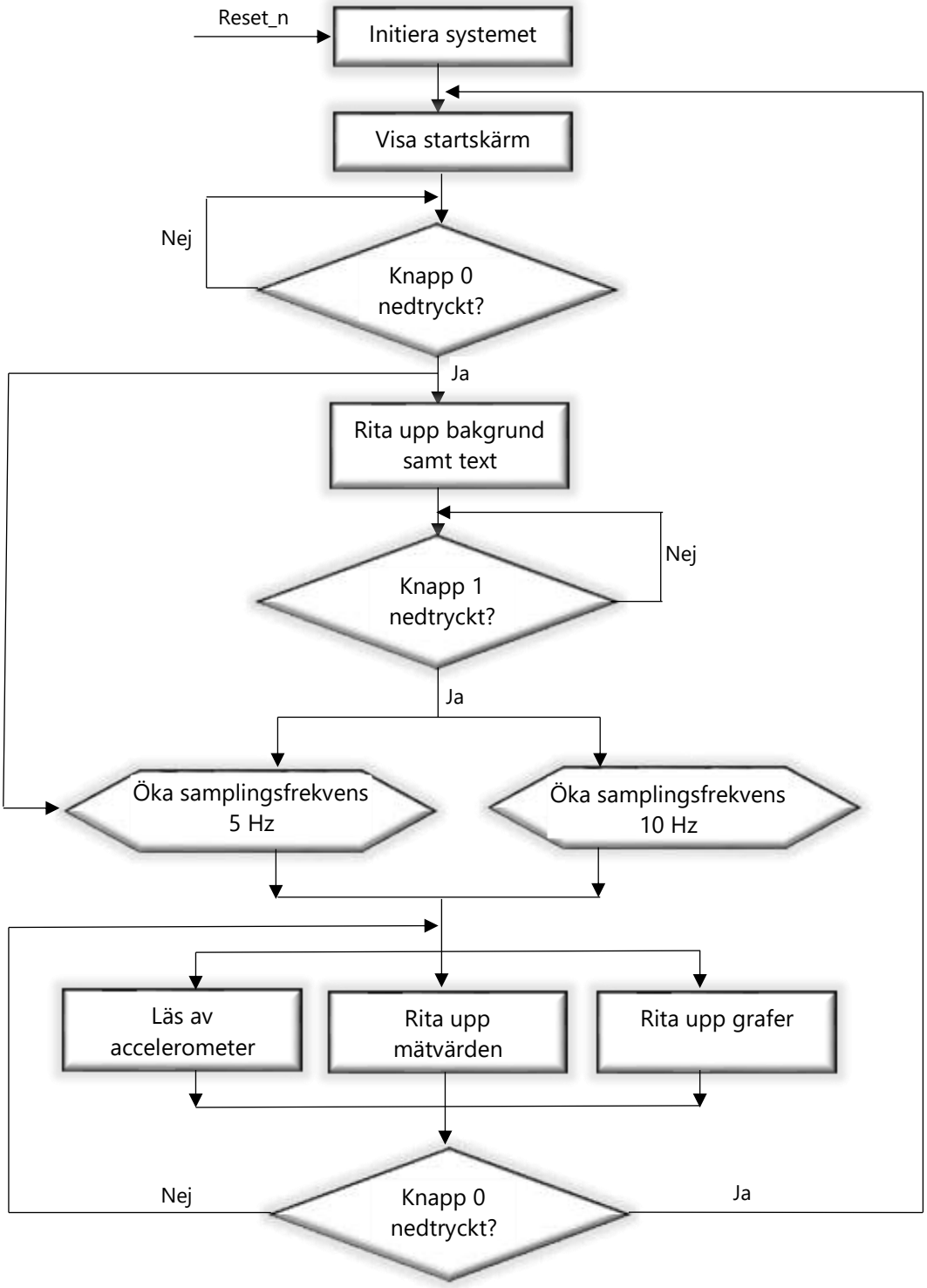


Figur 2. Blockschema De10-Lite [1]

8.1. Mjukvara

8.1.1. Systemarkitektur

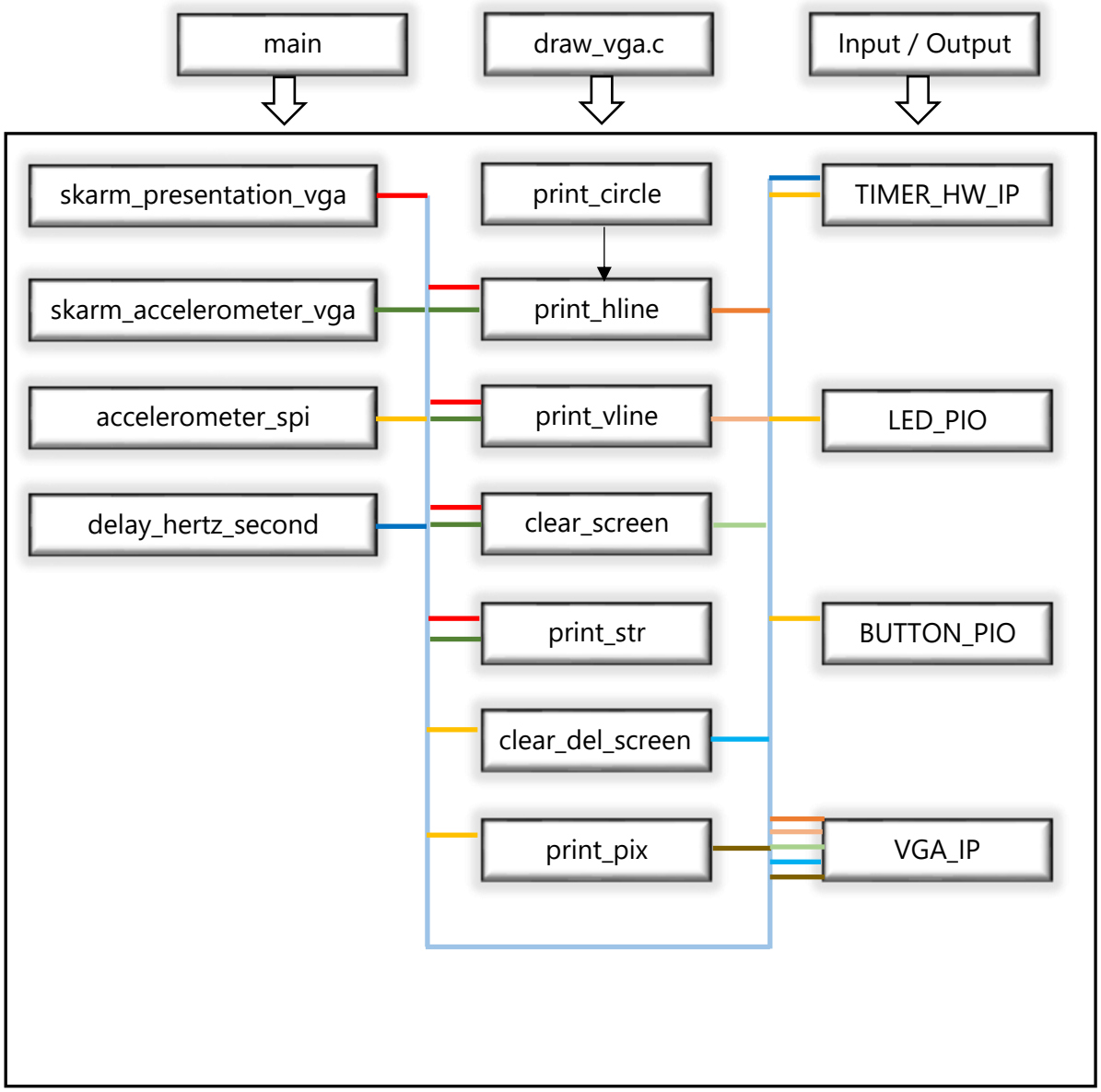
Flödesschemat för mjukvaran visas i Figur 3. När systemet börjar, visas en startskärm som beskriver hur man kontrollerar systemet. Efter att man tryckt på knapp 0 startar sedan en kontinuerlig sampling och uppdatering av mätvärdena på skärmen.



Figur 3. Flödesdiagram

8.1.2. Delsystem

Huvudprogrammet innehåller main och de funktioner som hanterar loggning av data, tidräkning och bildhantering. Funktionerna i draw_vga.c används av huvudprogrammet för att utföra de grundläggande grafik funktionerna som till exempel att tända pixlar, rita linjer och skriva tecken på skärmen. Grafikrutinerna använder i sin tur vga utgångens IP-komponent till att skriva och läsa i grafikminnet. De övriga I/O-komponenterna används av huvudprogrammet till att kommunicera med timern, lysdioderna och knapparna.



8.1.3. Drivrutiner

Det visar en översikt över de funktioner som krävdes av kravspecifikationen.

8.1.3.1. VGA färger

Värde (hexadecimalt)	Färg
0x00	
0x01	
0x02	
0x03	
0x04	
0x05	
0x06	
0x07	

8.1.3.2. VGA-drivrutiner

Tabell 5: "DE10_Lite_VGA_Driver.h" innehåller funktionsdeklarationer och inkluderas för att få tillgång till vga-rutinerna.

Tabell 6: ***"draw_vga.c"*** innehåller vga-rutinerna.

Tabell 7: ***"DE10_Lite_VGA_Driver.h"*** innehåller också definitioner av macron som fungerar för att läser och skriver värden i minnesplacering.

Tabell 5. Funktionsdeklarationer i DE10_Lite_VGA_Driver.h

Funktion	Beskrivning
void print_pix(alt_u32 x, alt_u32 y, alt_u8 rgb);	Skriver ut en pixel på koordinaterna x,y med färgen rgb.
void print_hline(alt_u32 x_start, alt_u32 y_start, alt_u32 length, alt_u8 rgb);	Skriver ut en horisontell linje med färgen rgb. Linjen börjar på x_start,y_start och är length pixlar lång.
void print_vline(alt_u32 x_start, alt_u32 y_start, alt_u32 length, alt_u8 rgb);	Skriver ut en vertikal linje med färgen rgb. Linjen börjar på x_start,y_start och är length pixlar lång.
void print_char(alt_u32 x, alt_u32 y, alt_u8 rgb, alt_u8 bg_rgb, alt_u8 ch);	Skriver ut ett tecken, ch, på x,y. Tecknet har färgen rgb och bakgrundsfärgen bg_rgb.
alt_u8 read_pixel_ram_int(alt_u32 x, alt_u32 y);	Läser av värdet på pixeln på koordinaten x,y, det vill säga färgvärdet.
void print_circle(alt_u32 radius, alt_u32 x, alt_u32 y, alt_u8 rgb);	Skriver ut en cirkel med radien radius och färgen rgb. Centrum är x,y.
void clear_screen(alt_u8 rgb);	Rensar skärmen med färgen rgb.

void print_str(alt_u32 x, alt_u32 y, alt_u8 rgb, alt_u8 bg_rgb, alt_u8 *str);	Skriver ut en sträng på koordinaterna x,y med färgen rgb och bakgrundsfärgen bg_rgb. Skulle strängen fortsätta utanför högerkanten fortsätter den på raden under. Skulle den gå utanför nederkanten fortsätter den längst upp till vänster.
void int_print(alt_32 x_start, alt_32 y_start, int data, int data_l, alt_32 color, alt_u32 BGcolor)	Skriver ut siffror(data) med antal siffror(data_l) på koordinaterna x,y med färgen color och bakgrundsfärgen BGrgb.

Tabell 6. VGA-rutinerna i draw_vga.c

Funktion	Beskrivning
print_pix(82,72,0x02) ;	i kolumn,rad skriver ut pixeln med färgen grön.
print_hline(0,14,282, 0x07);	ritar en 282 pixlar lång horisontell vit linje med start uppe i vänstra.
print_vline(7,4,220, 0x07);	ritar en 220 pixlar lång vertikalvit linje med start uppe i vänstra hörnet.
Print_char(10, 20, 0x04, 0x07, 'D');	skriver ut bokstaven ”D” i röd med vit bakgrund högst upp i vänstra hörnet.
alt_u8 var = read_pixel_ram_int(0,0);	returnerar färg koden på pixeln längst uppe i vänstra.
print_circle(40, 50, 50, 1);	ritar och fyller i en blå cirkel med en radie på 40 pixlar i övre vänstra hörnet av skärmen.
clear_screen(0x00);	skriver över alla adresser i bild minnet med färgen black.
print_str(8, 6, 1, ”TEIS”, 0x04,0);	skriver ut texten ”TEIS” i röd färg högst upp i vänstra hörnet på skärmen.
int_print(40,65,123456,3,0x02,0)	Skriver ut tre heltal med färgen grön

Tabell 7. VGA-macron i DE10_Lite_VGA_Driver.h

Funktion	Beskrivning
#define write_pixel(x,y,color) IOWR_32DIRECT(DE10_LITE_VGA_IP_0_BASE,((320*(y)+(x))*4),(color))	Skriver värde ”color” till position x_pos,y_pos i minnet.

#define read_pixel(x,y) IORD_32DIRECT(DE10_LITE_VGA_IP_0_BASE,(320*(y)+(x))*4)	Läser värde från x_pos, y_pos i minnet.

8.1.3.3. Timmer-drivrutiner

För att använda timer-drivrutinen finns de macron som är listade i Tabell 8.

Tabell 8. Timmer-drivrutiner i altera_avalon_timer_regs.h

Funktion	Beskrivning
#define TIMER_STOP IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x00000000)	Stoppar timern
#define TIMER_RESET IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x40000000)	Nollställer timern
#define TIMER_START IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x80000000)	Startar timern
#define TIMER_READ IORD_32DIRECT(TIMER_HW_IP_0_BASE,0)	Läser ut timervärdet

8.1.3.4. SPI-drivrutiner

Tabell 9: "altera_up_avalon_accelerometer_spi.h" innehåller funktionsdeklarationer och inkluderas för att få tillgång till accelerometer-rutinerna. ***"altera_up_avalon_accelerometer_spi.c"*** innehåller accelerometer-rutinerna.

"altera_up_avalon_accelerometer_spi_regs.h" innehåller definitioner av macron som fungerar för att läser och skriver värden i minnesplacering.

Tabell 9. Funktionsdeklarationer i altera_up_avalon_accelerometer_spi.h/.c

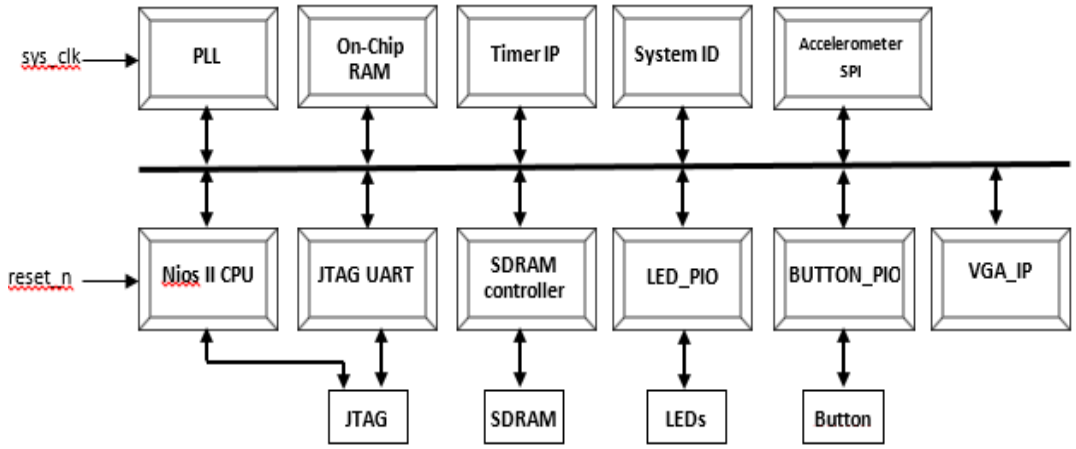
Funktion	Beskrivning
alt_up_accelerometer_spi_dev* alt_up_accelerometer_spi_open_dev(const char* name);	Öppnar "Accelerometer SPI Mode" enheten som anges av "namn"
int alt_up_accelerometer_spi_read_address_register(alt_up_accelerometer_spi_dev *accel_spi, alt_u8 *addr);	Läser konfigurationsdata från en av inbyggda videoenhet register.
int alt_up_accelerometer_spi_read(alt_up	Läser data från accelerometer register.

_accelerometer_spi_dev *accel_spi, alt_u8 addr, alt_u8 *data);	
int alt_up_accelerometer_spi_write(alt_up_accelerometer_spi_dev *accel_spi, alt_u8 addr, alt_u8 data);	Skriver data till accelerometer register.
int alt_up_accelerometer_spi_read_x_axis (alt_up_accelerometer_spi_dev *accel_spi, alt_32 *x_axis);	Läser X-axelvärdet från båda registren från accelerometer och omvandlar värdet till ett signed heltal.
int alt_up_accelerometer_spi_read_y_axis (alt_up_accelerometer_spi_dev *accel_spi, alt_32 *y_axis);	Läser Y-axelvärdet från båda registren från accelerometer och omvandlar värdet till ett signed heltal.
int alt_up_accelerometer_spi_read_z_axis (alt_up_accelerometer_spi_dev *accel_spi, alt_32 *z_axis);	Läser Z-axelvärdet från båda registren från accelerometer och omvandlar värdet till ett signed heltal.

8.2. Hårdvara

8.2.1. Systemarkitektur

Systemarkitekturen som FPGA:n programmerats med är en enkel bussarkitektur med en mikroprocessor av typen Nios II som styr, se Figur 5.



Figur 5. Systemarkitektur, blockschema FPGA

8.2.2. Minneskarta

Memory-map från ”sumary.html” för konstruktionen, se figur 6.

Slave Descriptor	Address Range	Size	Attributes
accelerometer_spi_0	0x080912D0 - 0x080912D1	2	
jtag_uart	0x080912C8 - 0x080912CF	8	printable
modular_adc_0_sequencer_csr	0x080912C0 - 0x080912C7	8	
TIMER_HW_IP_0	0x080912B0 - 0x080912BF	16	
pll_0	0x080912A0 - 0x080912AF	16	
pll_1	0x08091290 - 0x0809129F	16	
pio_leds_out	0x08091280 - 0x0809128F	16	
pio_buttons_in	0x08091270 - 0x0809127F	16	
pio_switches_in	0x08091260 - 0x0809126F	16	
DE10_Lite_SSD_IP_0	0x08091240 - 0x0809125F	32	
DE10_Lite_Arduino_IP_0	0x08091200 - 0x0809123F	64	
modular_adc_0_sample_store_csr	0x08091000 - 0x080911FF	512	
onchip_ram	0x08088000 - 0x0808FFFF	32768	memory
DE10_Lite_VGA_IP_0	0x08000000 - 0x0807FFFF	524288	
sdram_controller	0x04000000 - 0x07FFFFFF	67108864	memory
sysid_qsys_0	0x00000000 - 0x00000007	8	

Figur 6. Adressområden IP komponenter [2]

8.2.3. Delsystem

8.2.3.1. PLL

En PLL finns tillgänglig i systemet för att distribuera varianter av systemklockan på 50 MHz till delar av systemet.

8.2.3.2. On-Chip RAM : onchip_ram

Intern styrenhet för SDRAM-minnet.

8.2.3.3. Timer IP : TIMER_HW_IP_0

Timer komponent som mäter tid genom att räkna klockcykler.

8.2.3.4. System ID : sysid_qsys_0

En komponent som innehåller information om hårdvaruversion.

8.2.3.5. Accelerometer SPI : accelerometer_spi_0

Komponent som läser från accelerometern över SPI-bussen

8.2.3.6. JTAG UART : jtag_uart

Kommunikationsgränssnitt mellan USB Blaster och data/instruktionsbus. Används för kommunikation mellan PC och CPU för programmering och debuggning.

8.2.3.7. SDRAM Controller : sdram_controller

Extern styrenhet för SDRAM-minnet.

8.2.3.8. LED_PIO : pio_lkeds_out

Parallell I/O-komponent för lysdioderna.

8.2.3.9. BUTTON_PIO : pio_buttons_in

Parallell I/O-komponent för knapparna.

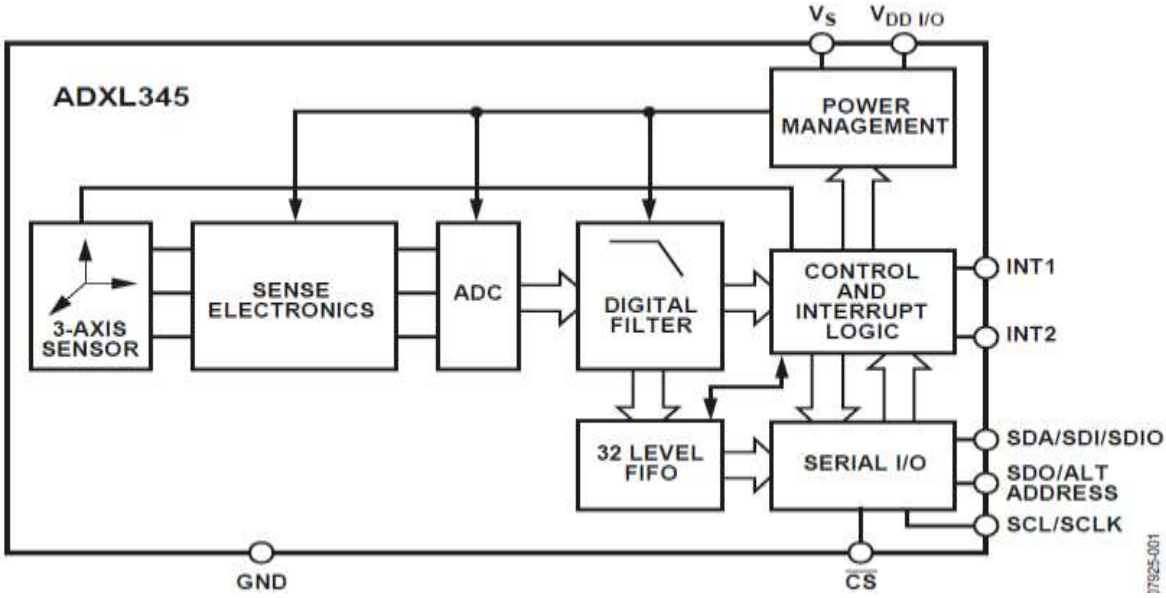
8.2.3.10. VGA IP : DE10_Lite_VGA_IP_0

En enkel 640x480 VGA grafikkontroller till 25 MHz.

8.3. Accelerometer (G-sensor)

Den Digital Accelerometer ADXL345 har I2C och SPI som kommunikationsprotokoll och är perfekt för alla 3V eller 5V mikrokontroller. Sensorn har tre mätaxlar (X Y Z), och stift som kan användas med buss gränssnittet I2C eller SPI. Du kan ställa in känslighetsnivån till + -2g, + -4g, + -8g eller + -16g. Det lägre intervallet ger högre upplösning för långsamma rörelser, medan det högre intervallet är utmärkt för spårning med hög hastighet.

Rör man sedan kortet i endera riktningen kommer accelerometern registrera de accelerationer som verkar på kortet, antingen positiva eller negativa värden.



Figur 7. Accelerometer funktionell blockschema [3]

9. VERIFIERING AV TESTPROTOKOLL

Tabell 10. Testprotokoll - verifiering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset av systemet. Hela skärmen skall rensas vid uppstart av programmet.	Funktionen clearScreen(0) används för att skriva färgvärdet svart (0) till hela grafikminnet.	Ja
2	Presentation av projektet visas efter att skärmen rensats vid uppstart.	Introduktionsskärm med en kort presentation visas efter uppstart.	Ja
3	Knapp 0 skall skifta mellan ”presentations”-läge och ”loggnings”-läge.	Skärmen visar presentations”-läge eller ”loggnings”-läge.	Ja
4	Accelerometer är vald efter presentations -läge och uppdateringsfrekvens 5 Hz är vald automatisk	Debugutskrift visar klockcykler som ger ca 5 Hz frekvens avläsning av accelerometer.	Ja

5	Tryck på knapp 1 en gång för att öka samplingsfrekvens 10 Hz.	Debugutskrift visar klockcykler som ger ca 10 Hz frekvens avläsning av accelerometer	Ja
6	Tryck på knapp 1 en gång för att minska samplingsfrekvens 5 Hz.	Debugutskrift visar klockcykler som ger ca 5 Hz frekvens avläsning av accelerometer.	Ja
7	Lysdioder LEDR3,LEDR2,LEDR, LEDR0	På kortet visa fyra lysdioder	Ja

10. VALIDERING AV TESTPROTOKOLL

En kort film som demonstrerar projektet finns på adressen.
https://www.youtube.com/watch?v=doeLQHqvuxI&t=28s

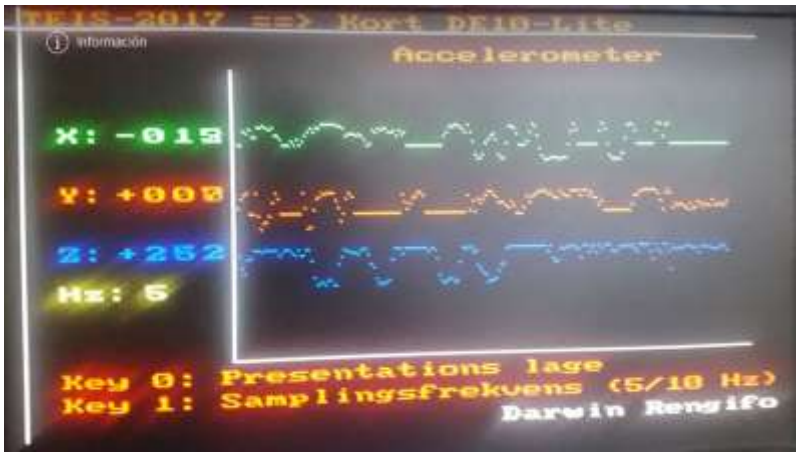
Tabell 11. Testprotokoll - validering

Testfall	Beskrivning	Acceptansbeskrivning	Validering
1	Presentationsbild visas vid uppstart.	Presentationsbilden ska finnas i VGA-minnet efter reset av systemet.	Ja (se figur 8)
2	Tryck på knapp 0 en gång för att välja "loggnings"-läge.	Det kommer att visa "loggnings"-läge.	Ja
3	Accelerometer är vald efter presentations -läge och uppdateringsfrekvens 5 Hz är vald automatisk.	Det visar på skärmen tre axlarna(X,Y,Z) som grafiska och siffror.	Ja (se figur 9)
4	Tryck på knapp 1 en gång för att öka samplingsfrekvens 10 Hz.	Kortet accelereras i respektive riktning (X,Y,Z) så plottas prickarna i grafen ut ovanför samt nedanför i varje axlar.	Ja (se figur 10)
5	Tryck på knapp 1 en gång för att öka samplingsfrekvens 5 Hz.	Kortet minska accelerationen i respektive riktning (X,Y,Z) så plottas prickarna i grafen ut ovanför samt nedanför i varje axlar.	Ja

6	Tryck på knapp 0 en gång för att välja “presentations”-läge.		Ja
7	Lysdioder LEDR3,LEDR2,LEDR, LEDR0	Fyra lysdioder som visar processorbelastning.	Ja (se figur 11)



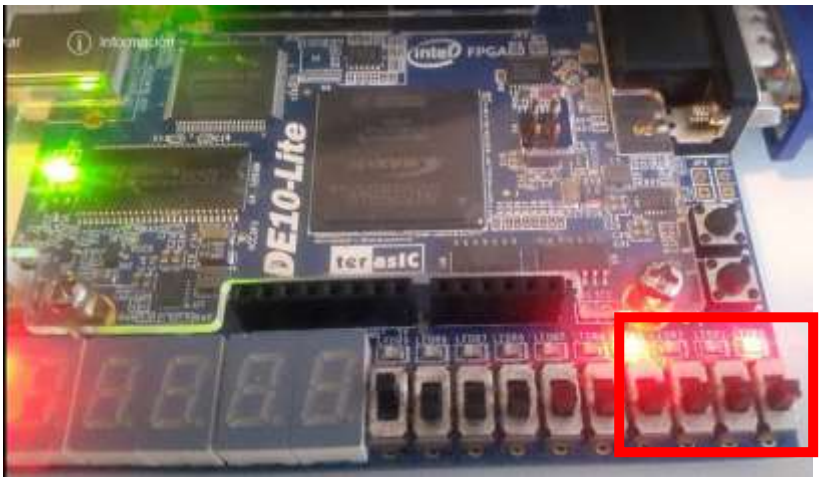
Figur 8. ***Presentationsskärm.***



Figur 9. Accelerometer 5Hz.



Figur 10. Accelerometer 10Hz.



Figur 11. Fyra lysdioder som visar processorbelastning.

11. ANALYS FÖRE OPTIMERING

Verktyget som användes för utveckling av var Nios II v17.0 Software Build Tools. Den kompilerade applikationskoden är ca 89956=15f64 Kb stor utan debug funktioner, som visas i figur 12 och figur 13.
Exakt tid för exekveringen beräknas enligt följande princip och förs till testprotokollet: Timerns klockfrekvens är 50MHz . (cc(clock cykel)*1)/50 = XXXns exekveringstid i ns som visar tabell 12 och tabell 13.

Storleken hos de olika sektionerna togs fram genom att läsa ur objdump-filen som skapas vid kompilering. Storleken på elf-filen togs fram genom kommandot ”nios2-elf-size”

```
$ nios2-elf-size C_engeenering.elf
   text    data     bss     dec     hex filename
  82540    7048     368   89956   15f64 C_engeenering.elf
```

Figur 12. Bestämning av total storlek.

```
Sections:
Idx Name          Size      VMA       LMA       File off  Algn
  0 .entry        00000020  08088000  08088000  00018000  2**5
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .exceptions   00000160  08088020  04000000  00001020  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  2 .text         00013374  04000160  04000160  00002160  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  3 .rodata       00000d78  040134d4  040134d4  000154d4  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .rwdata       00001b88  0401424c  04015dd4  0001624c  2**2
                  CONTENTS, ALLOC, LOAD, DATA, SMALL_DATA
  5 .bss          00000170  0401795c  0401795c  0001895c  2**2
                  ALLOC, SMALL_DATA
```

Figur 13. Bestämning av total storlek från objdump arkiv.

Tabell 12. Tidsåtgång - VGA-funktioner

Funktion	Tidsåtgång (offset = 50mhz) (cc*1)/50mhz = XXXns
clear_screen(BLACK);	109542
print_vline(7,4,220, WHITE);	4055
print_hline(0,14,282, WHITE);	726
print_str(8, 6, 1, ”TEIS”, RED,0);	5442
int_print (40,65,x_data,3,GREEN,0);	1108
print_pix(82,72,GREEN) ;	25840
clear_del_screen(82,35,270,180, BLACK);	98662

Tabell 13. Tidsåtgång - programfunktioner

Funktion	Tidsåtgång (offset = 50mhz) (cc*1)/50mhz
skarm_presentation_vga(void)	71992
skarm_accelerometer_vga(void)	144510
delay_hertz_second(int *param_hertz)	200006
accelerometer_spi(alt_u32 key_in_0)	70431

12. OPTIMERING

Som synes i tabell 14 anpassades BSP:n för att bli så effektiv och snabb som möjligt.

Tabell 14. BSP-Optimeringsparametrar använda.

Funktion	Tidsåtgång (offset = 50mhz) (cc*1)/50mhz
enable_small_c_library	På
enable_reduced_device_drivers	På
enable_lightweight_device_driver_api	På
enable_exit	Av
enable_clean_exit	Av
enable_c_plus_plus	Av
Optimize	Size

13. ANALYS EFTER OPTIMERING

Verktyget som användes för utveckling av var Nios II v17.0 Software Build Tools. Den kompilerade applikationskoden är ca 33472=82c0 Kb stor utan debug funktioner, optimering av koden visas i figur 14 och figur 15.
Exakt tid för exekveringen beräknas enligt följande princip och förs till testprotokollet: Timerns klockfrekvens är 50MHz . (cc(clock cykel)*1)/50 = XXXns exekveringstid i ns som visar tabell 15 och tabell 16.

Storleken hos de olika sektionerna togs fram genom att läsa ur objdump-filen som skapas vid kompilering. Storleken på elf-filen togs fram genom kommandot ”nios2-elf-size”

```
$ nios2-elf-size C_engeenering.elf
   text    data     bss     dec     hex filename
  32332     844     296   33472    82c0 C_engeenering.elf
```

Figur 14. Bestämning av total storlek.

```
Sections:
Idx Name          Size      VMA       LMA       File off  Algn
  0 .entry        00000020  08088000  08088000  0000b000  2**5
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .exceptions   00000160  08088020  04000000  00001020  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  2 .text         0000714c  04000160  04000160  00002160  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  3 .rodata       00000b80  040072ac  040072ac  000092ac  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .rwdata       0000034c  04007e2c  04008178  00009e2c  2**2
                  CONTENTS, ALLOC, LOAD, DATA, SMALL_DATA
  5 .bss          00000128  040084c4  040084c4  0000a4c4  2**2
                  ALLOC, SMALL_DATA
```

Figur 15. Bestämning av total storlek från objdump arkiv.

Tabell 15. Tidsåtgång - VGA-funktioner

Funktion	Tidsåtgång (offset = 50mhz) (cc*1)/50mhz = XXXns
clear_screen(BLACK);	109540
print_vline(7,4,220, WHITE);	4053
print_hline(0,14,282, WHITE);	723
print_str(8, 6, 1, ”TEIS”, RED,0);	5467
int_print (40,65,x_data,3,GREEN,0);	1087
print_pix(82,72,GREEN) ;	25773
clear_del_screen(82,35,270,180, BLACK);	90576

Tabell 16. Tidsåtgång - programfunktioner

Funktion	Tidsåtgång (offset = 50mhz) (cc*1)/50mhz
skarm_presentation_vga(void)	71546
skarm_accelerometer_vga(void)	144604
delay_hertz_second(int *param_hertz)	200001
accelerometer_spi(alt_u32 key_in_0)	247695

14. SAMMANFATTNING

Detta dokument beskriver utvecklingen av en prototyp av ett system som logga kretskortets accelerometer DE10-Lite, dessutom för att vetta alla större steg i en utvecklingsprocess. I rapporten behandlas krav som testprotokoll, verifiering, analys, optimering och validering.
Mikrodatornjukvaran exekveras ett gränssnitt för användaren att se värden från accelerometer, inbyggda i plattformen, på en VGA-skärm.
Användaren kan välja uppdateringsfrekvens i två steg: 5 Hz, 10 Hz. Dessa val görs med knapparna på plattformen och för att visa processorbelastning det använder lysdioderna(3...0).
Det har skapat drivrutiner som optimerar rengöringen av en sektor på VGA-skärmen och drivrutiner för att ändra värden med samplingsfrekvenser 5 eller 10Hz i realtid.

Kända buggar: När projektet kompileras uppkommer det några varningar, ingen av dessa hindrar dock att projektet kompilerar klart. Detta antal varningar som från kompilatorn som verkar vara knutna till att de definierade datatyperna i funktionerna.

main.c:142:19: warning: pointer targets in passing argument 3 of 'print_str' differ in signedness [-Wpointer-sign]

main.c:297:12: warning: format '%d' expects argument of type 'int', but argument 2 has type 'alt_u32 {aka long unsigned int}' [-Wformat=]

Framtida förbättringar: Framtida förbättringar av systemet som kan göras är att på grafikuppritningen, visar pixel separat när värderna ändras. Förbättrar definierade datatyperna för att undvika kompilatorn varningarna.

15. REFERENSER

[1] DE10-Lite_User_Manual.pdf
[2] C:\..... \TEIS\ELCURSO\C_kurs\c_eng_jobb\software\c_eng_jobb_bsp\summary.html
[3] Manual ADXL345.pdf
[4] Nioss II classic software developers_Handbook
[5] https://www.andrew.cmu.edu/course/60-257/reference/TWO_PI.html
[6] http://dunadigital.com/processing/2013/07/18/dibujar-una-onda-con-point-y-sin/